

Relatório de Auditoria de Segurança

Obra 10

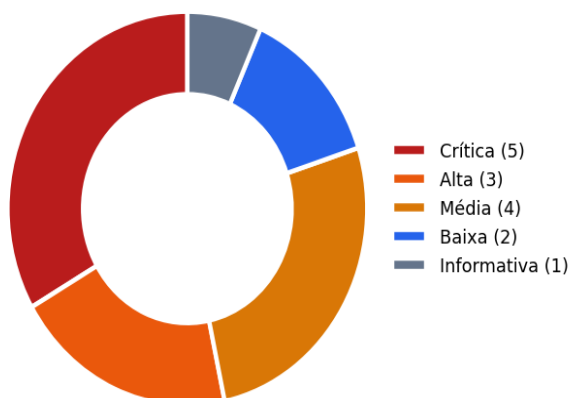
Data: 21/09/2026 · Escopo: backend NestJS, frontend React, Prisma, scripts rastreados, Dockerfile, GitHub Actions e histórico Git.

Nota metodológica. Stack detectada: TypeScript / NestJS + Prisma (sem RLS do Postgres/Supabase); auth JWT em cookie HttpOnly + Bearer; isolamento por **empresald no JWT** + **ObraContextGuard (x-obra-id)** + SuperAdminGuard + CapabilitiesService; frontend Vite/React; deploy Docker + Railway via GitHub Actions (sem Helm/Terraform). Categorias mapeadas assim: (1) queries/listagens sem filtro de empresald/obra; (2) UI esconde por papel vs guard/capability no handler; (3) todos os controllers *.controller.ts percorridos por ID em path/query/body; (4) defaults JWT/PayPal/ENCRYPTION_KEY, scripts commitados e bundle VITE_*; (5) dangerouslySetInnerHTML/eval no React e interpolação HTML no e-mail. Só entram achados com arquivo e linha no código atual.

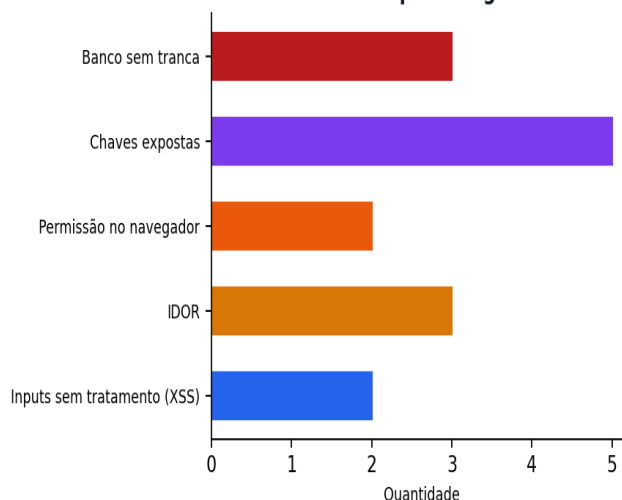
1. Resumo executivo

Foram confirmados **15 achados** no código: **5 críticos**, 3 altos, 4 médios, 2 baixos e 1 informativo. Os riscos centrais são takeover de tenant via e-mail compartilhado, confirmação falsa de pagamento PayPal, JWT forjável com default público e credenciais de produção no Git.

Achados por severidade



Achados por categoria



CRÍTICA

ALTA

MÉDIA

BAIXA

INFORMATIVA

1.1 Pontos fortes (verificados)

■ **Isolamento de RDO/anexos/alertas de listagem:** RdoController, AnexosController e AiController (relatório) usam JwtAuthGuard + ObraContextGuard; o guard confere obra.empresald === usuario.empresald (obra-context.guard.ts:57–59).

■ **CRUD de usuários da empresa:** UsuariosController exige gerenciarUsuarios e os services filtram findFirst({ id, empresald }) em update/delete/reenviar-convite (usuarios.controller.ts:32–96; usuarios.service.ts:265–266, 345–346).

■ **Catálogo, features, cupom de validação e cobranças GET:** catalogo.controller, feature.controller, cupom validar e cobranca obter/status/link-cartao/aplicar-cupom amarram empresald do JWT.

■ **Painel admin:** admin-empresas, admin-usuarios, admin-modulos, admin-cupons, admin-financeiro, admin-metricas e admin-features usam @UseGuards(JwtAuthGuard, SuperAdminGuard) no controller.

■ **Webhook Asaas:** rejeita se ASAAS_WEBHOOK_TOKEN vazio ou divergente (cobranca.controller.ts:159–172). CSRF exclui essa rota de propósito.

■ **Upload de logo/obra:** logo exige empresald === :id (ou SUPER_ADMIN) + gerenciarEmpresa; imagem de obra checa obra.empresald (upload.controller.ts:204–308).

■ **Frontend React:** zero ocorrências de dangerouslySetInnerHTML, innerHTML, eval ou new Function em src/. AdminRoute esconde /admin se perfil ≠ SUPER_ADMIN — e o backend tem SuperAdminGuard (o gap é o claim não ser relido no DB).

■ **Usuários — capability no servidor:** mesmo /gestor/usuarios sendo só ProtectedRoute no React, o backend recusa sem gerenciarUsuarios.

■ **Defesas transversais:** Helmet+CSP, CSRF double-submit (exceto login/register/webhook/mcp), bcrypt + lockout 10/15 min, jwtVersion ao trocar senha, SanitizePipe global no body, ThrottlerGuard.

■ **CI/Docker:** Dockerfile sem segredos; workflow usa `${ secrets.RAILWAY_TOKEN }`. Bundle frontend só lê `VITE_API_URL / VITE_SUPPORT_WHATSAPP` (não há chave de pagamento no cliente).

■ **MCP/Luna:** `McpController` exige JWT e as tools filtram `empresald`; listar usuários/financeiro na Luna checa capabilities.

1.2 Pontos fracos (riscos centrais)

■ O isolamento de tenant é **manual** (`JWT.empresald`), não RLS. Qualquer handler que esqueça o filtro vaza. O login já fura o modelo ao tratar e-mail como identidade global.

■ Billing tem um interruptor de fraude: `PayPal mock + confirmarPagamentoLocal`.

■ O default JWT + SuperAdmin só no token é um caminho direto a painel global se a env falhar.

■ O Git contém connection string de produção e senhas de `SUPER_ADMIN` — isso é incidente ativo, não só dívida técnica.

2. Achados detalhados por categoria

Cada linha foi lida no arquivo indicado. Trechos abaixo são cópia fiel (segredos de banco/senha mascarados neste PDF).

2.1 Banco sem tranca

Sev.	Arquivo:linha	Descrição
CRÍTICA	obra10-backend/src/modules/auth/auth.service.ts:86–101	F-01 — Senha de uma empresa destrava todas as contas do mesmo e-mail. Se o e-mail existe em várias empresas, qualquer senha que bata em UMA conta libera o picker de TODAS e, com empresald, emite JWT da conta-alvo sem a senha dessa conta.
CRÍTICA	obra10-backend/src/modules/cobranca/paypal.service.ts:21–24 e 88–90	F-02 — PayPal em modo MOCK confirma pagamento real. Sem PAYPAL_CLIENT_ID/SECRET o serviço entra em mockMode e captureOrder sempre devolve COMPLETED. O controller então chama confirmarPagamentoLocal e marca a cobrança como PAGA.
CRÍTICA	obra10-backend/src/modules/cobranca/cobranca.controller.ts:132–146	F-03 — Capture PayPal grava PAGO após status COMPLETED (inclusive mock). O handler não distingue mock de captura real. Qualquer COMPLETED (forjado pelo mock) ativa módulos.

2.2 Permissão no navegador

Sev.	Arquivo:linha	Descrição
ALTA	obra10-backend/src/modules/tenant/tenant.controller.ts:65–88	F-06 — Leitura de plano e cobranças sem gerenciarFinanceiro. GET tenants/meu-plano e GET tenants/meu-plano/cobrancas só exigem JWT + empresald. upgrade/desativar exigem a capability; a leitura não. O frontend esconde Financeiro com canManageFinanceiro, mas /gestor/financeiro e /assinatura são só ProtectedRoute.
INFORMATIVA	obra10-backend/src/core/guards/super-admin.guard.ts:15–17	F-15 — SuperAdminGuard confia só no claim JWT, não no banco. Compara user.perfilGlobal !== 'SUPER_ADMIN' no payload. Não relê o perfil no Prisma. JwtAuthGuard só revalida jwtVersion e ativo.

2.3 IDOR

Sev.	Arquivo:linha	Descrição
ALTA	obra10-backend/src/modules/upload/upload.controller.ts:256–272	F-07 — Upload de foto altera qualquer usuário por ID. Quem tem gerenciarUsuarios (ou é dono do próprio id) pode PATCH a foto de qualquer UUID. Não há filtro empresald no alvo. O update devolve o objeto usuario.
ALTA	obra10-backend/src/app.controller.ts:90–108	F-08 — GET /debug-fs lista o filesystem sem autenticação. Endpoint público devolve cwd, lista de arquivos do diretório de trabalho e contents de uploads/. Sem JwtAuthGuard.
MÉDIA	obra10-backend/src/modules/rdo/alerta.controller.ts:44–49	F-09 — Marcar alerta lido sem checar obra/tenant. PATCH alertas/:id/marcar-lido faz update por id puro. O controller tem ObraContextGuard (exige x-obra-id válido), mas o alerta atualizado pode ser de outra obra/tenant.

2.4 Chaves expostas

Sev.	Arquivo:linha	Descrição
CRÍTICA	obra10-backend/src/core/guards/jwt-auth.guard.ts:29–30	F-04 — Segredo JWT com default público e sem rejeição no startup. verifyAsync usa process.env.JWT_SECRET 'obra10-mvp-secret-key-12345'. O mesmo default está em auth.module.ts:14. Não há validação de boot que recuse o default.
CRÍTICA	obra10-backend/set_simple_admin_pass.js:4–16	F-05 — Connection string de produção e senha de superadmin no Git. Scripts versionados contém URL PostgreSQL do Railway (centerbeam.proxy.rlwy.net:19827) com senha do postgres, e definem senhas de superadmin@obra10.com (Lunardeli123 / Lunardeli20011978\$). Também em update_superadmin_pass.js, scratch/sync_db_prod_to_local.js e scratch/update_superadmin_password.js.
MÉDIA	obra10-backend/src/core/services/crypto.service.ts:36–38	F-11 — ENCRYPTION_KEY ausente grava PII em claro. Se ENCRYPTION_KEY não existe, o serviço só loga warning e armazena CPF/telefone sem cifra. /health ainda revela se a chave está definida e o tamanho.
MÉDIA	obra10-backend/prisma/seed.ts:136 e 187–204	F-12 — Seed redefine senha de tarcisio@lunardeli.com.br para Senha123. bcrypt.hash('Senha123') e upsert com update: { senhaHash: hashTarcisio }. Rodar seed contra um banco que já tem a empresa Lunardeli sobrescreve a senha real.
BAIXA	obra10-backend/src/app.controller.ts:54–65	F-14 — /health público vaza metadados de configuração. Sem auth. Expõe nodeEnv, buildId, se ENCRYPTION_KEY existe e o length, se ASAAS_API_KEY/WEBHOOK_TOKEN estão configurados e o environment Asaas.

2.5 Inputs sem tratamento (XSS)

Sev.	Arquivo:linha	Descrição
MÉDIA	obra10-backend/src/modules/email/email.service.ts:139, 214–215, 457–466	F-10 — HTML de e-mail interpola input sem escape. nomeEmpresa, motivo de rejeição de RDO, nome/telefone/mensagem do formulário público /contato entram em template HTML sem escape. SanitizePipe (regex) remove script/iframe/on*, mas deixa tags como , <a>, <svg> e entidades.
BAIXA	obra10-backend/src/modules/upload/upload.controller.ts:42	F-13 — Upload aceita image/svg+xml (XSS armazenado). ALLOWED_IMAGE_TYPES inclui svg+xml. O arquivo vai ao R2 público com Content-Type do cliente. Abrir a URL crua no browser executa script do SVG.

3. Fichas dos achados (arquivo por arquivo)

CRÍTICA
F-01 Senha de uma empresa destrava todas as contas do mesmo e-mail

Onde: obra10-backend/src/modules/auth/auth.service.ts:86–101 · **Categoria:** Banco sem tranca

O que é: Se o e-mail existe em várias empresas, qualquer senha que bata em UMA conta libera o picker de TODAS e, com empresaId, emite JWT da conta-alvo sem a senha dessa conta.

Por que é explorável: Explorável hoje: o produto permite o mesmo e-mail em várias empresas. Um colaborador convidado em A com senha própria acessa B (ex.: gestor) sem conhecer a senha de B.

Condição: Mesmo e-mail em ≥ 2 empresas ativas.

```
if (empresaId) {
  const alvo = candidatos.find((c) => c.empresaId === empresaId);
  // alvo vem de TODOS os candidatos, não só dos matched
  return this.emitirSessao(alvo.id);
}
return { precisaEscolherEmpresa: true, empresas: candidatos.map(...) };
```

CRÍTICA
F-02 PayPal em modo MOCK confirma pagamento real

Onde: obra10-backend/src/modules/cobranca/paypal.service.ts:21–24 e 88–90 · **Categoria:** Banco sem tranca

O que é: Sem PAYPAL_CLIENT_ID/SECRET o serviço entra em mockMode e captureOrder sempre devolve COMPLETED. O controller então chama confirmarPagamentoLocal e marca a cobrança como PAGA.

Por que é explorável: Qualquer usuário autenticado do tenant chama POST /cobrancas/:id/paypal/capture-order e ativa o plano sem pagar. Em produção o cartão foi migrado para Asaas; as rotas PayPal continuam vivas.

Condição: PAYPAL_CLIENT_ID ou PAYPAL_SECRET ausentes (padrão atual).

```
this.mockMode = !this.clientId || !this.secret;
async captureOrder(orderId: string) {
  if (this.mockMode) return { status: 'COMPLETED' };
}
```

CRÍTICA
F-03 Capture PayPal grava PAGO após status COMPLETED (inclusive mock)

Onde: obra10-backend/src/modules/cobranca/cobranca.controller.ts:132–146 · **Categoria:** Banco sem tranca

O que é: O handler não distingue mock de captura real. Qualquer COMPLETED (forjado pelo mock) ativa módulos.

Por que é explorável: Encadeia com F-02: um POST autenticado basta para fraude de billing.

Condição: Mesma de F-02.

```
const captureResult = await this.paypal.captureOrder(orderId);
if (captureResult.status === 'COMPLETED') {
  await this.cobrancaService.confirmarPagamentoLocal(cobrancaId, req.user.empresaId);
}
```

CRÍTICA
F-04 Segredo JWT com default público e sem rejeição no startup

Onde: obra10-backend/src/core/guards/jwt-auth.guard.ts:29–30 · **Categoria:** Chaves expostas

O que é: verifyAsync usa process.env.JWT_SECRET || 'obra10-mvp-secret-key-12345'. O mesmo default está em auth.module.ts:14. Não há validação de boot que recuse o default.

Por que é explorável: Quem conhece o default (está no Git) forja JWT com perfilGlobal SUPER_ADMIN. SuperAdminGuard só lê o claim do token, não o banco.

Condição: Explorável se JWT_SECRET não estiver definido no ambiente. Residual: default conhecido no histórico Git.

```
const payload = await this.jwtService.verifyAsync(token, {
  secret: process.env.JWT_SECRET || 'obra10-mvp-secret-key-12345',
});
```

CRÍTICA**F-05** Connection string de produção e senha de superadmin no Git

Onde: obra10-backend/set_simple_admin_pass.js:4–16 · **Categoria:** Chaves expostas

O que é: Scripts versionados contém URL PostgreSQL do Railway (centerbeam.proxy.rlwy.net:19827) com senha do postgres, e definem senhas de superadmin@obra10.com (Lunardeli123 / Lunardeli20011978\$). Também em update_superadmin_pass.js, scratch/sync_db_prod_to_local.js e scratch/update_superadmin_password.js.

Por que é explorável: Credencial de banco + senha de SUPER_ADMIN no histórico público do repositório. Quem clona o repo (ou lê o Git) acessa o banco e a conta admin.

Condição: Arquivos rastreados pelo Git (confirmado por git ls-files). A senha do proxy Railway deve ser considerada vazada.

```
const passwordToSet = 'Lunardeli123';
const client = new Client({
  connectionString: "postgresql://postgres:***REDACTADO***@centerbeam.proxy.rlwy.net:19827/railway",
});
UPDATE usuarios SET senha_hash = $1 WHERE email = 'superadmin@obra10.com'
```

ALTA**F-06** Leitura de plano e cobranças sem gerenciarFinanceiro

Onde: obra10-backend/src/modules/tenant/tenant.controller.ts:65–88 · **Categoria:** Permissão no navegador

O que é: GET tenants/meu-plano e GET tenants/meu-plano/cobrancas só exigem JWT + empresaId. upgrade/desativar exigem a capability; a leitura não. O frontend esconde Financeiro com canManageFinanceiro, mas /gestor/financeiro e /assinatura são só ProtectedRoute.

Por que é explorável: USER/EXTERNO autenticado chama a API e lê faturamento do tenant. POST cobranças/contratar (cobranca.controller.ts:59–64) também não checa a capability — qualquer usuário do tenant gera cobrança.

Condição: Usuário autenticado do tenant sem gerenciarFinanceiro.

```
@Get('tenants/meu-plano')
async getMeuPlano(@Req() req: any) {
  const empresaId = req.user?.empresaId;
  if (!empresaId) throw new ForbiddenException('Sessão inválida.');
```

```
  return this.tenantService.obterMeuPlano(empresaId);
}
```

ALTA**F-07** Upload de foto altera qualquer usuário por ID

Onde: obra10-backend/src/modules/upload/upload.controller.ts:256–272 · **Categoria:** IDOR

O que é: Quem tem gerenciarUsuarios (ou é dono do próprio id) pode PATCH a foto de qualquer UUID. Não há filtro empresaId no alvo. O update devolve o objeto usuario.

Por que é explorável: Gestor do tenant A, conhecendo/adivinando o UUID, sobrescreve a foto de usuário do tenant B (ou de SUPER_ADMIN) e recebe dados do registro.

Condição: Capability gerenciarUsuarios ou SUPER_ADMIN; UUID da vítima.

```
const podeGerenciar = isAdmin
  ? true
  : await this.capabilities.hasCapability(req.user.sub, 'gerenciarUsuarios');
if (!isOwner && !podeGerenciar) throw new ForbiddenException(...);
const usuario = await this.prisma.usuario.update({
  where: { id }, data: { fotoUrl: url },
});
```

ALTA

F-08 GET /debug-fs lista o filesystem sem autenticação**Onde:** obra10-backend/src/app.controller.ts:90–108 · **Categoria:** IDOR

O que é: Endpoint público devolve cwd, lista de arquivos do diretório de trabalho e contents de uploads/. Sem JwtAuthGuard.

Por que é explorável: Reconhecimento de deploy, paths internos e arquivos em disco (incluindo uploads locais em não-produção).

Condição: Rota alcançável na API publicada.

```
@Get('debug-fs')
debugFs() {
  return {
    cwd, cwdFiles: fs.readdirSync(cwd),
    uploadsPath, uploadsFiles, dirname: __dirname,
  };
}
```

MÉDIA

F-09 Marcar alerta lido sem checar obra/tenant**Onde:** obra10-backend/src/modules/rdo/alerta.controller.ts:44–49 · **Categoria:** IDOR

O que é: PATCH alertas/:id/marcar-lido faz update por id puro. O controller tem ObraContextGuard (exige x-obra-id válido), mas o alerta atualizado pode ser de outra obra/tenant.

Por que é explorável: Usuário com acesso a qualquer obra marca como lido um alerta de outro tenant se souber o UUID (integridade; impacto limitado).

Condição: JWT + x-obra-id de uma obra própria + UUID do alerta alheio.

```
@Patch('/:id/marcar-lido')
async marcarLido(@Param('id') id: string) {
  return this.prisma.alertaObra.update({
    where: { id }, data: { lido: true },
  });
}
```

MÉDIA

F-10 HTML de e-mail interpola input sem escape**Onde:** obra10-backend/src/modules/email/email.service.ts:139, 214–215, 457–466 · **Categoria:** Inputs sem tratamento (XSS)

O que é: nomeEmpresa, motivo de rejeição de RDO, nome/telefone/mensagem do formulário público /contato entram em template HTML sem escape. SanitizePipe (regex) remove script/iframe/on*, mas deixa tags como , <a>, <svg> e entidades.

Por que é explorável: Formulário público (contato.controller.ts:28–31) envia HTML ao inbox de suporte. XSS armazenado no cliente de e-mail / phishing via <a href>. Motivo de rejeição de RDO é controlado pelo gestor.

Condição: SanitizePipe global no body; bypass por tags não listadas. Destinatário abre HTML.

```
<p><strong>Nome:</strong> ${dto.nome}</p>
<p style="white-space:pre-wrap">${dto.mensagem}</p>
<p style="margin:8px 0 0">${motivo}</p>
```

MÉDIA

F-11 ENCRYPTION_KEY ausente grava PII em claro**Onde:** obra10-backend/src/core/services/crypto.service.ts:36–38 · **Categoria:** Chaves expostas

O que é: Se ENCRYPTION_KEY não existe, o serviço só loga warning e armazena CPF/telefone sem cifra. /health ainda revela se a chave está definida e o tamanho.

Por que é explorável: Dump de banco ou backup expõe documentos fiscais. health (app.controller.ts:54–65) ajuda o atacante a saber se a cifra está off.

Condição: ENCRYPTION_KEY não configurada no processo.

```
} else {
  this.logger.warn(
    'ENCRYPTION_KEY não configurada. Campos sensíveis serão armazenados SEM criptografia.',
  );
}
```

MÉDIA

F-12 Seed redefine senha de `tarcisio@lunardeli.com.br` para `Senha123`

Onde: `obra10-backend/prisma/seed.ts:136 e 187–204` · **Categoria:** Chaves expostas

O que é: `bcrypt.hash('Senha123')` e `upsert` com `update: { senhaHash: hashTarcisio }`. Rodar seed contra um banco que já tem a empresa Lunardeli sobrescreve a senha real.

Por que é explorável: Script versionado + `DATABASE_URL` de produção = takeover da conta gestor. `Senha123` também é a senha demo de `engenheiro@acme.com`.

Condição: Execução do seed apontando para banco não-local.

```
const hashTarcisio = await bcrypt.hash('Senha123', 10);
update: { senhaHash: hashTarcisio, ativo: true, deletedAt: null },
email: 'tarcisio@lunardeli.com.br',
```

BAIXA

F-13 Upload aceita `image/svg+xml` (XSS armazenado)

Onde: `obra10-backend/src/modules/upload/upload.controller.ts:42` · **Categoria:** Inputs sem tratamento (XSS)

O que é: `ALLOWED_IMAGE_TYPES` inclui `svg+xml`. O arquivo vai ao R2 público com `Content-Type` do cliente. Abrir a URL crua no browser executa script do SVG.

Por que é explorável: Gestor com `gerenciarEmpresa` sobe logo SVG malicioso; quem abre o objeto (não só `<img>`) sofre XSS na origem do CDN.

Condição: Upload de logo/foto com MIME SVG; vítima navega à URL do R2.

```
const ALLOWED_IMAGE_TYPES = /^image\/(jpeg|jpg|png|gif|webp|svg\+xml|heic|heif)$/i;
```

BAIXA

F-14 `/health` público vaza metadados de configuração

Onde: `obra10-backend/src/app.controller.ts:54–65` · **Categoria:** Chaves expostas

O que é: Sem auth. Expõe `nodeEnv`, `buildId`, se `ENCRYPTION_KEY` existe e o `length`, se `ASAAS_API_KEY`/`WEBHOOK_TOKEN` estão configurados e o `environment` `Asaas`.

Por que é explorável: Reconhecimento para decidir se JWT default, cifra ou webhook estão off.

Condição: Sempre explorável (rota pública).

```
hasEncryptionKey: process.env.ENCRYPTION_KEY
  ? `defined_len_${process.env.ENCRYPTION_KEY.length}` : 'undefined',
asaas: { configured: Boolean(...), environment: ..., webhookToken: Boolean(...) }
```

INFORMATIVA

F-15 SuperAdminGuard confia só no claim JWT, não no banco

Onde: `obra10-backend/src/core/guards/super-admin.guard.ts:15–17` · **Categoria:** Permissão no navegador

O que é: Compara `user.perfilGlobal !== 'SUPER_ADMIN'` no payload. Não relê o perfil no Prisma. `JwtAuthGuard` só revalida `jwtVersion` e ativo.

Por que é explorável: Não é falha sozinha: o backend TEM o gate. Vira crítica se o JWT for forjado (F-04) ou se o usuário for rebaixado sem incrementar `jwtVersion`.

Condição: Token com claim `SUPER_ADMIN` (forjado ou stale).

```
if (!user || user.perfilGlobal !== 'SUPER_ADMIN') {
  throw new ForbiddenException('Acesso restrito a super administradores...');
}
```

4. Recomendações priorizadas

P1**Rotacionar agora**

Trocar senha do postgres Railway, de superadmin@obra10.com e de qualquer conta que tenha usado Lunardeli123 / Lunardeli20011978\$ / Senha123. Revogar o proxy público se ainda estiver aberto. Remover os scripts do Git (incluir no .gitignore) e considerar git filter-repo no histórico.

P1**Fechar o login multi-empresa**

emitirSessao só para contas em matched. O picker só lista empresas cuja senha bateu. empresald no body deve apontar para um matched, nunca para candidatos crus.

P1**Matar o PayPal mock em produção**

Se PAYPAL_* vazio: recusar create/capture com 503. Preferível remover as rotas se o checkout é só Asaas. Nunca chamar confirmarPagamentoLocal sem captura verificada no provedor.

P1**JWT_SECRET obrigatório**

Falhar o bootstrap se JWT_SECRET ausente, < 32 chars ou igual a obra10-mvp-secret-key-12345. SuperAdminGuard deve ler perfilGlobal no banco.

P2**Capability financeira no servidor**

GET meu-plano, histórico, listar cobranças e POST contratar exigem gerenciarFinanceiro (exceto SUPER_ADMIN). Espelhar o gate do frontend.

P2**IDOR de foto e alerta**

Upload de foto: alvo.empresald === req.user.empresald. Alerta: update where { id, obraId } do contexto.

P2**Desligar debug-fs e enxugar /health**

Remover debug-fs do binário de produção. Health só status/db, sem length de chave nem flags Asaas.

P3**Escape HTML em e-mails + banir SVG em logo**

Função escapeHtml em todos os interpolados. Recusar image/svg+xml no upload de imagem. Endurecer SanitizePipe (ou DOMPurify no servidor) — regex sozinho não é sanitizer.

P3**ENCRYPTION_KEY obrigatória em production**

process.exit(1) se NODE_ENV=production e a chave não for hex de 64 chars. Seed nunca deve update senha de contas reais.

5. Issues para o GitHub

Cada bloco abaixo está pronto para copiar e colar numa issue. Delimitadores --- ISSUE n --- / --- FIM ISSUE n ---. Achados do mesmo tema foram agrupados para não gerar spam. Senhas e connection strings aparecem mascaradas; o arquivo no repo tem o valor real — rode o rotate antes de abrir a issue em repositório público.

Issue 1 — [Segurança] Login com e-mail compartilhado emite sessão de outra empresa sem a senha dela

CRÍTICA

```
--- ISSUE 1 ---
# [Segurança] Login com e-mail compartilhado emite sessão de outra empresa sem a senha dela

**Labels sugeridas:** `security, crítica`

## Problema
Se o e-mail existe em várias empresas, qualquer senha que bata em UMA conta libera o picker de TODAS e, com empresaId, emite JWT.

## Por que é explorável
Explorável hoje: o produto permite o mesmo e-mail em várias empresas. Um colaborador convidado em A com senha própria acessa B.

## Evidência
**F-01** `obra10-backend/src/modules/auth/auth.service.ts:86-101`
...
if (empresaId) {
  const alvo = candidatos.find((c) => c.empresaId === empresaId);
  // alvo vem de TODOS os candidatos, não só dos matched
  return this.emitirSessao(alvo.id);
}
return { precisaEscolherEmpresa: true, empresas: candidatos.map(...) };
...

## Impacto
Explorável hoje: o produto permite o mesmo e-mail em várias empresas. Um colaborador convidado em A com senha própria acessa B.

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] login com senha da empresa A NÃO lista nem emite sessão da empresa B
- [ ] empresaId no body só é aceito se a senha bateu naquela conta
- [ ] teste automatizado cobre e-mail em 2 empresas com senhas distintas

--- FIM ISSUE 1 ---
```

Issue 2 — [Segurança] Capture PayPal em modo MOCK marca cobrança como PAGA

CRÍTICA

```
--- ISSUE 2 ---
# [Segurança] Capture PayPal em modo MOCK marca cobrança como PAGA

**Labels sugeridas:** `security, crítica`

## Problema
Sem PAYPAL_CLIENT_ID/SECRET o serviço entra em mockMode e captureOrder sempre devolve COMPLETED. O controller então chama confi...

## Por que é explorável
Qualquer usuário autenticado do tenant chama POST /cobrancas/:id/paypal/capture-order e ativa o plano sem pagar. Em produção o...

## Evidência
**F-02** `obra10-backend/src/modules/cobranca/paypal.service.ts:21-24 e 88-90`
...
this.mockMode = !this.clientId || !this.secret;
async captureOrder(orderId: string) {
  if (this.mockMode) return { status: 'COMPLETED' };
}
...
**F-03** `obra10-backend/src/modules/cobranca/cobranca.controller.ts:132-146`
...
const captureResult = await this.paypal.captureOrder(orderId);
```

```

if (captureResult.status === 'COMPLETED') {
  await this.cobrancaService.confirmarPagamentoLocal(cobrancaId, req.user.empresaId);
}
...

## Impacto
Qualquer usuário autenticado do tenant chama POST /cobrancas/:id/paypal/capture-order e ativa o plano sem pagar. Em produção o

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] sem credenciais PayPal, create/capture devolvem 503 e NÃO alteram cobrança
- [ ] confirmarPagamentoLocal só após captura real verificada no provedor
- [ ] rotas PayPal desligadas se o produto usa só Asaas

--- FIM ISSUE 2 ---

```

Issue 3 — [Segurança] JWT_SECRET com default público e SuperAdminGuard só no claim

CRÍTICA

```

--- ISSUE 3 ---
# [Segurança] JWT_SECRET com default público e SuperAdminGuard só no claim

**Labels sugeridas:** `security, crítica`

## Problema
verifyAsync usa process.env.JWT_SECRET || 'obra10-mvp-secret-key-12345'. O mesmo default está em auth.module.ts:14. Não há vali

## Por que é explorável
Quem conhece o default (está no Git) forja JWT com perfilGlobal SUPER_ADMIN. SuperAdminGuard só lê o claim do token, não o banco

## Evidência
**F-04** `obra10-backend/src/core/guards/jwt-auth.guard.ts:29-30`
...
const payload = await this.jwtService.verifyAsync(token, {
  secret: process.env.JWT_SECRET || 'obra10-mvp-secret-key-12345',
});
...
**F-15** `obra10-backend/src/core/guards/super-admin.guard.ts:15-17`
...
if (!user || user.perfilGlobal !== 'SUPER_ADMIN') {
  throw new ForbiddenException('Acesso restrito a super administradores...');
}
...

## Impacto
Quem conhece o default (está no Git) forja JWT com perfilGlobal SUPER_ADMIN. SuperAdminGuard só lê o claim do token, não o banco

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] processo recusa subir se JWT_SECRET ausente, curto ou igual ao default
- [ ] SuperAdminGuard relê perfilGlobal no banco
- [ ] default removido do código e do histórico (rotate se já usou o default)

--- FIM ISSUE 3 ---

```

Issue 4 — [Segurança] Credenciais de produção e senhas de admin commitadas no Git

CRÍTICA

```

--- ISSUE 4 ---
# [Segurança] Credenciais de produção e senhas de admin commitadas no Git

**Labels sugeridas:** `security, crítica`

## Problema
Scripts versionados contém URL PostgreSQL do Railway (centerbeam.proxy.rlwy.net:19827) com senha do postgres, e definem senhas
## Por que é explorável

```

Credencial de banco + senha de SUPER_ADMIN no histórico público do repositório. Quem clona o repo (ou lê o Git) acessa o banco

Evidência

```
**F-05** `obra10-backend/set_simple_admin_pass.js:4-16`
...

const passwordToSet = 'Lunardeli123';
const client = new Client({
  connectionString: "postgresql://postgres:***REDACTADO***@centerbeam.proxy.rlwy.net:19827/railway",
});
UPDATE usuarios SET senha_hash = $1 WHERE email = 'superadmin@obra10.com'
...

**F-12** `obra10-backend/prisma/seed.ts:136 e 187-204`
...

const hashTarcisio = await bcrypt.hash('Senha123', 10);
update: { senhaHash: hashTarcisio, ativo: true, deletedAt: null },
email: 'tarcisio@lunardeli.com.br',
...

## Impacto
Credencial de banco + senha de SUPER_ADMIN no histórico público do repositório. Quem clona o repo (ou lê o Git) acessa o banco

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/secreto no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] scripts com connection string/senhas removidos do Git e do histórico
- [ ] senha do postgres Railway, superadmin e contas citadas rotacionadas
- [ ] seed NÃO sobrescreve senha de usuário real; Senha123 só em fixture isolada

--- FIM ISSUE 4 ---
```

Issue 5 — [Segurança] Endpoints financeiros não exigem gerenciarFinanceiro

ALTA

--- ISSUE 5 ---

[Segurança] Endpoints financeiros não exigem gerenciarFinanceiro

Labels sugeridas: `security, alta`

Problema

GET tenants/meu-plano e GET tenants/meu-plano/cobrancas só exigem JWT + empresaId. upgrade/desativar exigem a capability; a lei

Por que é explorável

USER/EXTERN0 autenticado chama a API e lê faturamento do tenant. POST cobranças/contratar (cobranca.controller.ts:59-64) também

Evidência

```
**F-06** `obra10-backend/src/modules/tenant/tenant.controller.ts:65-88`
...

@Get('tenants/meu-plano')
async getMeuPlano(@Req() req: any) {
  const empresaId = req.user?.empresaId;
  if (!empresaId) throw new ForbiddenException('Sessão inválida.');
```

Impacto

USER/EXTERN0 autenticado chama a API e lê faturamento do tenant. POST cobranças/contratar (cobranca.controller.ts:59-64) também

Sugestão de correção

- Corrigir o fluxo no arquivo citado para validar posse/secreto no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

Critérios de aceite

- [] GET meu-plano, histórico, listar cobranças e POST contratar exigem gerenciarFinanceiro (ou GESTOR)
- [] USER/EXTERN0 recebe 403; teste de API cobre isso

--- FIM ISSUE 5 ---

Issue 6 — [Segurança] IDOR no upload de foto de usuário (sem filtro de tenant)

ALTA

--- ISSUE 6 ---

[Segurança] IDOR no upload de foto de usuário (sem filtro de tenant)

Labels sugeridas: `security, alta`

Problema

Quem tem gerenciarUsuarios (ou é dono do próprio id) pode PATCH a foto de qualquer UUID. Não há filtro empresaId no alvo. O up

Por que é explorável

Gestor do tenant A, conhecendo/adivinando o UUID, sobrescreve a foto de usuário do tenant B (ou de SUPER_ADMIN) e recebe dados

Evidência

F-07 `obra10-backend/src/modules/upload/upload.controller.ts:256-272`

...

const podeGerenciar = isAdmin

 ? true

 : await this.capabilities.hasCapability(req.user.sub, 'gerenciarUsuarios');

if (!isOwner && !podeGerenciar) throw new ForbiddenException(...);

const usuario = await this.prisma.usuario.update({

 where: { id }, data: { fotoUrl: url },

});

...

Impacto

Gestor do tenant A, conhecendo/adivinando o UUID, sobrescreve a foto de usuário do tenant B (ou de SUPER_ADMIN) e recebe dados

Sugestão de correção

- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.

- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.

- Rotacionar qualquer credencial que tenha sido commitada.

Critérios de aceite

- [] update de foto exige usuario.empresaId === req.user.empresaId (salvo SUPER_ADMIN)

- [] UUID de outro tenant retorna 403/404

--- FIM ISSUE 6 ---

Issue 7 — [Segurança] Endpoints públicos debug-fs e health vazam superfície interna

ALTA

--- ISSUE 7 ---

[Segurança] Endpoints públicos debug-fs e health vazam superfície interna

Labels sugeridas: `security, alta`

Problema

Endpoint público devolve cwd, lista de arquivos do diretório de trabalho e contents de uploads/. Sem JwtAuthGuard.

Por que é explorável

Reconhecimento de deploy, paths internos e arquivos em disco (incluindo uploads locais em não-produção). Reconhecimento para de

Evidência

F-08 `obra10-backend/src/app.controller.ts:90-108`

...

@Get('debug-fs')

debugFs() {

 return {

 cwd, cwdFiles: fs.readdirSync(cwd),

 uploadsPath, uploadsFiles, dirname: __dirname,

 };

}

...

F-14 `obra10-backend/src/app.controller.ts:54-65`

...

hasEncryptionKey: process.env.ENCRYPTION_KEY

 ? `defined\_len\_\${process.env.ENCRYPTION\_KEY.length}` : 'undefined',

asaas: { configured: Boolean(...), environment: ..., webhookToken: Boolean(...) }

...

Impacto

Reconhecimento de deploy, paths internos e arquivos em disco (incluindo uploads locais em não-produção).

Sugestão de correção

- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

```
## Critérios de aceite
- [ ] GET /debug-fs removido ou atrás de SuperAdminGuard + NODE_ENV!==production
- [ ] /health não expõe length de chave nem flags de provedor de pagamento

--- FIM ISSUE 7 ---
```

Issue 8 — [Segurança] IDOR ao marcar alerta como lido sem amarrar à obra

MÉDIA

```
--- ISSUE 8 ---
# [Segurança] IDOR ao marcar alerta como lido sem amarrar à obra

**Labels sugeridas:** `security, média`

## Problema
PATCH alertas/:id/marcar-lido faz update por id puro. O controller tem ObraContextGuard (exige x-obra-id válido), mas o alerta

## Por que é explorável
Usuário com acesso a qualquer obra marca como lido um alerta de outro tenant se souber o UUID (integridade; impacto limitado).

## Evidência
**F-09** `obra10-backend/src/modules/rdo/alerta.controller.ts:44-49`
...
@Patch('/:id/marcar-lido')
async marcarLido(@Param('id') id: string) {
  return this.prisma.alertaObra.update({
    where: { id }, data: { lido: true },
  });
}
...

## Impacto
Usuário com acesso a qualquer obra marca como lido um alerta de outro tenant se souber o UUID (integridade; impacto limitado).

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] update de alerta filtra obraId (do guard) e empresa da obra
- [ ] UUID de outro tenant não altera linha

--- FIM ISSUE 8 ---
```

Issue 9 — [Segurança] HTML de e-mail e upload SVG sem sanitização adequada

MÉDIA

```
--- ISSUE 9 ---
# [Segurança] HTML de e-mail e upload SVG sem sanitização adequada

**Labels sugeridas:** `security, média`

## Problema
nomeEmpresa, motivo de rejeição de RDO, nome/telefone/mensagem do formulário público /contato entram em template HTML sem escap

## Por que é explorável
Formulário público (contato.controller.ts:28-31) envia HTML ao inbox de suporte. XSS armazenado no cliente de e-mail / phishing

## Evidência
**F-10** `obra10-backend/src/modules/email/email.service.ts:139, 214-215, 457-466`
...
<p><strong>Nome:</strong> ${dto.nome}</p>
<p style="white-space:pre-wrap">${dto.mensagem}</p>
<p style="margin:8px 0 0">${motivo}</p>
...
**F-13** `obra10-backend/src/modules/upload/upload.controller.ts:42`
...
const ALLOWED_IMAGE_TYPES = /^image\/(jpeg|jpg|png|gif|webp|svg\+xml|heic|heif)$/i;
...
```

```

## Impacto
Formulário público (contato.controller.ts:28-31) envia HTML ao inbox de suporte. XSS armazenado no cliente de e-mail / phishing

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] todos os campos interpolados em e-mail passam por escape HTML
- [ ] SVG rejeitado no upload de imagem (ou servido com Content-Type que não executa script)

--- FIM ISSUE 9 ---

```

Issue 10 — [Segurança] Startup aceita ENCRYPTION_KEY ausente e grava PII em claro

MÉDIA

```

--- ISSUE 10 ---
# [Segurança] Startup aceita ENCRYPTION_KEY ausente e grava PII em claro

**Labels sugeridas:** `security, média`

## Problema
Se ENCRYPTION_KEY não existe, o serviço só loga warning e armazena CPF/telefone sem cifra. /health ainda revela se a chave está off

## Por que é explorável
Dump de banco ou backup expõe documentos fiscais. health (app.controller.ts:54-65) ajuda o atacante a saber se a cifra está off

## Evidência
**F-11** `obra10-backend/src/core/services/crypto.service.ts:36-38`
...
} else {
  this.logger.warn(
    'ENCRYPTION_KEY não configurada. Campos sensíveis serão armazenados SEM criptografia.',
  );
}
...

## Impacto
Dump de banco ou backup expõe documentos fiscais. health (app.controller.ts:54-65) ajuda o atacante a saber se a cifra está off

## Sugestão de correção
- Corrigir o fluxo no arquivo citado para validar posse/segredo no servidor.
- Adicionar teste de regressão que falhe se o default inseguro ou o filtro ausente voltar.
- Rotacionar qualquer credencial que tenha sido commitada.

## Critérios de aceite
- [ ] produção recusa boot sem ENCRYPTION_KEY hex de 64 chars
- [ ] teste de startup cobre o reject

--- FIM ISSUE 10 ---

```

Fim do relatório. Cobertura: 23 arquivos *controller.ts do src (mais app.controller), guards de auth/obra/admin, auth.service, cobranca/paypal, e-mail, upload, seed, scripts rastreados, Dockerfile, .github/workflows/deploy-railway.yml, frontend src (XSS e rotas) e git ls-files/log -S. Não foram inventados achados fora desses arquivos.